

설계과제 1

○ 개요

- 정답 파일을 기준으로 학생들이 제출한 답안 파일을 채점하는 프로그램인 ssu_score의 분석 및 성능 개선

○ 목표

- 유닉스/리눅스 컴퓨팅 환경에서 제공하는 파일 입출력, 파일속성 디렉토리에 관한 시스템 호출 함수와 라이브러리 함수를 이용한 프로그램을 분석하고 새로운 프로그램을 작성하여 시스템 프로그램 설계 및 응용 능력을 향상

○ 팀 구성

- 개인별 프로젝트

○ 보고서 제출 방법 (강의계획서와 동일. #2 설계과제 명세에는 보고서 제출 방법을 삭제 예정. 강의계획서 참고)

- 설계과제는 “1_학번_버전번호.zip” (예. 1_20250000_V1.zip 또는 P1_2025000_V1.0.zip 등)형태로 압축하여 classroom.google.com에 제출해야 함.
 - 압축 파일 내 (1)구현보고서인 설계과제번호.hwp (예. 2.hwp 또는 2.doc) (2)헤더파일 및 소스코드 등 해당 디렉토리에서 컴파일과 실행이 가능하도록 모든 파일(makefile, obj, *.c, *.h 등 컴파일하고 실행하기 위한 파일)을 포함시켜야 함. 단, 특정한 디렉토리에서 실행해야 할 경우는 디렉토리는 포함하지 않아 됨.
 - 구현보고서인 “설계과제번호.hwp”에는 1. 과제개요(명세에서 주어진 개요를 그대로 쓰면 안됨. 자기가 분석하고, 구현한 내용 요약) 2. 기능(전체 기능 및 함수별 기능, 모든 함수의 프로토타입, 함수간 call graph (다양한 툴 이용 가능)), 3. 상세설계(기존 코드에서 수정할 부분 분석, 개선되어야 하는 이유, 설계 내용, 수정하거나 구현한 함수의 프로토타입 및 기능), 4. 실행 결과 (기존 코드 실행 결과와 수정한 코드의 실행 결과 비교 등), 5. 소스코드 (절대 캡처 안됨, 텍스트 수준으로 복사하고 반드시 주석 포함)를 반드시 포함시켜야 함.
 - 제출한 압축 파일을 풀었을 때 해당 디렉토리에서 컴파일 및 실행이 되어야 함 (모든 프로그램은 해당 디렉토리에서 컴파일 및 실행이 되어야 함(특정한 디렉토리에서 실행해야 할 경우는 제외). 해당 디렉토리에서 컴파일이나 실행되지 않을 경우, 설계과제 제출 방법(파일명, 컴파일 시에 포함되어야 할 파일에 포함되어야 할 파일 등)을 따르지 않는 경우 해당 과제 성적은 10점 감점.)
 - 과제를 기한 내 새로 제출할 경우 기존 것은 삭제하지 않고 P설계과제번호_학번_V2.zip 형태로 버전번호를 증가시키면 됨. 버전 번호는 대문자 V와 함께 integer를 1부터 incremental 증가시키면서 부여하면 됨. (예. V1, V2, V3, ...). 첫 제출 시에도 버전 번호 V1(또는 V1.0)를 붙여야 하며, 두 번째는 V2(또는 V2.0)를 붙여 제출하면 됨. 단, P설계과제번호_학번_V1.zip 압축 파일 내 구현보고서에는 버전번호는 붙이지 말 것.
- ### ○ 배점 기준 (#1번 과제는 분석이 포함되기 때문에 배점이 다소 다름)
- **(필수) 소스코드 분석 (30점)** : 설계과제번호.hwp (분석 및 개선 보고서)를 기준으로 채점. 개요 1점, 기능 1점, 상세설계 25점, 실행 결과 3점. 단, 실행되지 않는 소스코드를 제출할 경우 무조건 0점 처리. 즉 개선하거나 수정하지 않는 소스코드는 제출하지 말 것. 보고서 내 소스코드에 주석이 부실하거나 없을 경우 10점 감점. **소스코드 분석은 25점 이상을 받아야 필수를 다한 것으로 판단할 것임.**
 - 개선된 소스코드 (70점) : 기능 개선의 수월성(기준과 동일한 답안을 넣고, 동일한 채점 기준으로 시간 측정, 상위 5% 70점, 상위 10% 68점, 상위 20% 62점, 상위 30% 54점, 상위 40% 46점, 상위 50% 38점, 상위 60% 30점, 기타 25점 이하 부여. 단, 실행되지 않는 소스코드를 제출할 경우 무조건 0점 처리. 즉 개선하거나 수정하지 않는 소스코드는 제출하지 말 것. 프로그램 전체에서 system() 절대 사용 불가. 사용 시 본 과제는 0점 처리
 - (참고) 각 설계과제의 완성 유무는 제출 여부로 판단하는 것이 아니라 주어진 과제에서 명시된 “필수구현”의 구현 여부로 판단. 예. 특정 과제의 필수구현 중 일부만 구현했을 경우 해당 점수는 부여하나 과제는 미구현으로 판단하고 본 교과목 이수조건인 설계과제 최소 구현 개수 1개에 포함시키지 않음
 - 샘플 정답지(Correct_Answers)를 기준으로 샘플 답지(STD_Answers) 파일을 모두 자동 채점
 - 동일한 유형의 문제/답안이 자동 채점이 되어야 하기 때문에 학생들이 스스로 샘플 정답지와 샘플 답지 다양하게 만

들어 자동 채점.

○ 샘플 문제

※ 주어진 프로그램 실행 시 아래 실행결과가 나올 수 있도록, 빈 칸을 채우시오. [1-3, 총 15점]

(주의할 점 : (1) 답을 저장할 파일의 확장자는 .txt이며 각 문제 번호와 소문제 번호(문제마다 네모 박스에 기입된 번호)를 이용하여 파일을 생성해야 함 (2) 각 문제당 동일한 소문제번호가 있을 경우 한 개의 파일만 생성하면 됨. 예를 들어, 1번 문제는 (1) ~ (5) 번의 5개 소문제가 있기 때문에 각 1~3문제에 대해서 1-1.txt, 1-2.txt, 1-3.txt, 1-4.txt, 1-5.txt / 2-1.txt, 2-2.txt, 2-3.txt, 2-4.txt, 2-5.txt / 3-1.txt, 3-2.txt, 3-3.txt, 3-4.txt, 3-5.txt 등 5개의 각각 생성해야 함. 단, 틀린 답이나 답을 모르는 경우에도 파일을 생성해야 함.

아래 샘플 문제에서는 빨간색 부분이 정답이며, 위에서부터 1-1, 1-2...이며, 학생들에게 주어진 샘플 정답지와는 다른 것임.

1. 다음 프로그램은 dup2()를 호출하여 표준 출력 1번 파일 디스크립터를 4번으로 복사하고 4번 파일 디스크립터를 인자로 하여 write()를 호출하면 표준 출력에 쓰는 것과 같은 효과를 보인다. 아래 실행결과를 보고 빈칸을 채우시오.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define BUFFER_SIZE 1024

int main(void)
{
    char buf[BUFFER_SIZE];
    char *fname = "ssu_test.txt";
    int fd;
    int length;

    if ((fd = open(fname, O_RDONLY, 0644)) < 0) {
        fprintf(stderr, "open error for %s\n", fname);
        exit(1);
    }

    if (dup2(1, 4) != 4) {
        fprintf(stderr, "dup2 call failed\n");
        exit(1);
    }

    while (1) {
        length = read(fd, buf, BUFFER_SIZE);

        if (length <= 0)
            break;

        write(4, buf, length);
    }

    exit(0);
}
```

실행결과

```
root@localhost:/home/oslab# ./ssu_dup2_2
```

```
Linux System Programming!  
Unix System Programming!  
Linux Mania  
Unix Mania
```

2. 다음 프로그램은 rename()을 호출하여 파일의 이름을 ssu_test1.txt에서 ssu_test2.txt로 변경 하여 두 번째 open()이 실패하는 것을 보여준다. 아래 실행결과를 보고 빈칸을 채우시오.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <fcntl.h>  
  
int main(int argc, char *argv[])  
{  
    int fd;  
  
    if (argc != 3) {  
        fprintf(stderr, "usage: %s <oldname> <newname>\n", argv[0]);  
        exit(1);  
    }  
  
    if ((fd = open(argv[1], O_RDONLY)) < 0) {  
        fprintf(stderr, "first open error for %s\n", argv[1]);  
        exit(1);  
    }  
    else  
        close(fd);  
  
    if (rename(argv[1], argv[2]) < 0) {  
        fprintf(stderr, "rename error\n");  
        exit(1);  
    }  
  
    if ((fd = open(argv[1], O_RDONLY)) < 0)  
        printf("second open error for %s\n", argv[1]);  
    else {  
        fprintf(stderr, "it's very strange!\n");  
        exit(1);  
    }  
  
    if ((fd = open(argv[2], O_RDONLY)) < 0) {  
        fprintf(stderr, "third open error for %s\n", argv[2]);  
        exit(1);  
    }  
  
    printf("Everything is good!\n");  
    exit(0);  
}
```

실행결과

```
root@localhost:/home/oslab# vi ssu_test1.txt  
root@localhost:/home/oslab# ./ssu_rename ssu_test1.txt ssu_test2.txt
```

```
second open error for ssu_test1.txt
Everything is good!
root@localhost:/home/oslab# ls ssu_test2.txt
ssu_test2.txt
```

3. 다음 프로그램은 setvbuf()를 사용하여 버퍼를 조작하는 것을 보여준다. setvbuf()를 테스트하기 앞서 tty 명령어를 통해 터미널의 번호를 확인한다. tty 명령어는 현재 표준입력에 접속된 터미널 장치 파일 이름을 출력하는 명령어로 현재 장치 파일 이름을 알아낸 후, 해당 장치의 버퍼를 조작한다. 아래 프로그램을 실행하기 위해 현재 버퍼인 /dev/pts/19를 열고, setvbuf()를 설정하는 ssu_setbuf()를 호출한다. ssu_setbuf()의 첫 번째 인자인 파일에 대해 버퍼를 설정하고, 해당 파일의 파일 디스크립터를 얻는다. 얻은 파일 디스크립터를 통해 그에 맞는 모드와 크기를 설정한 후 setvbuf()를 호출하여 해당 파일의 버퍼를 설정한다. "Hello, UNIX!!" 출력은 버퍼가 설정된 후 실행되기 때문에 버퍼에 넣은 후 한 번에 출력하고, "HOW ARE YOU?" 출력은 버퍼가 NULL로 설정된 후 실행되기 때문에 fprintf()를 호출할 때마다 버퍼에 넣지 않고 바로 출력한다. 아래 실행결과를 보고 빈칸을 채우시오.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

void ssu_setbuf(FILE *fp, char *buf);

int main(void)
{
    char buf[BUFFER_SIZE];
    char *fname = "/dev/pts/19";
    FILE *fp;

    if ((fp = fopen(fname, "w")) == NULL) {
        fprintf(stderr, "fopen error for %s", fname);
        exit(1);
    }

    ssu_setbuf(fp, buf);
    fprintf(fp, "Hello, ");
    sleep(1);
    fprintf(fp, "UNIX!!");
    sleep(1);
    fprintf(fp, "\n");
    sleep(1);
    ssu_setbuf(fp, NULL);
    fprintf(fp, "HOW");
    sleep(1);
    fprintf(fp, " ARE");
    sleep(1);
    fprintf(fp, " YOU?");
    sleep(1);
    fprintf(fp, "\n");
    sleep(1);
    exit(0);
}
```

```

void ssu_setbuf(FILE *fp, char *buf) {
    size_t size;
    int fd;
    int mode;

    fd = fileno(fp);

    if (isatty(fd))
        mode = _IOLBF;
    else
        mode = _IOFBF;

    if (buf == NULL) {
        mode = _IONBF;
        size = 0;
    }
    else
        size = BUFFER_SIZE;

    setvbuf(fp, buf, mode, size);
}

```

실행결과

```

oslab@localhost:~$ tty
/dev/pts/19
oslab@localhost:~$ ./ssu_setvbuf
Hello, UNIX!!
HOW ARE YOU?

```

4. 다음은 프로그램을 gcc로 컴파일 후 gdb로 디버깅 하는 과정을 보여주고 있다. 빈칸의 gdb의 적절한 명령어를 쓰시오.

```

#include <stdio.h>
#define NUM 5

int score[NUM];

int sum(int cnt){
    int i;
    int sum;

    for(i = 0; i < cnt ; i++){
        sum += score[i];
    }
    return sum;
}

int main()
{
    int i = 0;
    int cnt = 0;

    printf("input scores. input -1 to finish.\n");

    for(i = 0; i < NUM; i++) {
        printf("score #%d : ", cnt+1);
        scanf("%d", &score[cnt]);
        if(score[cnt] == -1)

```

```

        break;
        cnt++;
    }

    printf("%d scores read.\n", cnt);
    printf("---- result ----\n");
    printf("sum : %d avg : %d\n", sum(cnt), sum(cnt)/cnt);

    return 0;
}

```

컴파일 및 gdb 실행 결과

```

~/source/gdb$ gcc -Wall -W -g bug.c
bug.c: In function 'main':
bug.c:25:9: warning: format '%d' expects argument of type 'int *', but argument 2 has type 'int'
[-Wformat=]
    scanf("%d", score[cnt]);
          ^
~/source/gdb$
~/source/gdb$ ./a.out
input scores. input -1 to finish.
score #1 : 102
세그멘테이션 오류 (core dumped)
~/source/gdb$
~/source/gdb$ gdb a.out -q
Reading symbols from a.out...done.
(gdb) r
Starting program: /home/oslab/source/gdb/a.out
input scores. input -1 to finish.
score #1 : 102

Program received signal SIGSEGV, Segmentation fault.
_I0_vfscanf_internal (s=0xb7fbb5a0 <_I0_2_1_stdin_>, format=0x804865f "%d", argptr=0xbffff604 "", errp=0x0)
    at vfscanf.c:1902
1902  vfscanf.c: 그런 파일이나 디렉터리가 없습니다.
(gdb) bt
#0  _I0_vfscanf_internal (s=0xb7fbb5a0 <_I0_2_1_stdin_>, format=0x804865f "%d", argptr=0xbffff604 "",
    errp=0x0) at vfscanf.c:1902
#1  0xb7e6513e in __isoc99_scanf (format=0x804865f "%d") at isoc99_scanf.c:37
#2  0x08048520 in main () at bug.c:25
(gdb) q
A debugging session is active.

        Inferior 1 [process 3812] will be killed.

Quit anyway? (y or n) y

```

- 제출 기한 : 4월 12일(일) 오후 11시 59분 59초
- 주의할 점 : 기존 소스코드의 함수 이름과 변수 이름은 변경 불가. 새로운 함수 및 변수 추가 가능. 예외 처리 부분 절대 삭제 불가. 예외 처리 부분 추가는 가능.